Data-Driven Test Automation Approach

Business Reference Document

Prepared for business stakeholders and project sponsors
Date: 13 October 2026

1. Executive Summary

The Clarien test automation repository is being moved from a **hard-coded, script-centric model** to an **Excel-driven, menu-driven model**. The goal is to let business testers and QA analysts configure test data and test flows from a familiar workbook rather than changing source code.

This document explains what the repository looked like before, what has been changed so far, the resulting benefits, the remaining limitations, and how five specific client requirements can be addressed. It is intended for business users and management, so technical detail is kept at a high level.

2. Previous State

Before the data-driven approach, the automation framework had the following characteristics:

- **Test data was embedded in the test scripts.** Account numbers, customer names, amounts, currency, and menu codes were written directly inside the TypeScript spec files.
- **Execution order was hard-coded.** The list of spec files that made up a business flow was stored in `tests/config/testOrder.json`. Adding, removing, or reordering a step required a code change.
- **Each new scenario often required a new or copied spec.** When a tester wanted a different account number or a different customer, the typical path was to edit the script or create a copy.
- **Shared state existed in code but was not driven by a configurable runner.** Values such as a new account ID or transaction ID could already be passed between specs through existing helpers, but the test data and flow ordering still had to be changed in source files.
- **Running a flow required command-line knowledge.** Business users could not easily pick a flow from a list and run it.

In short, the framework was powerful but not self-service for business testers. Small data changes still went through the development team.

3. What Has Been Modified

The following changes were introduced to build the Excel and menu-driven approach:

3.1 Single Excel Workbook

A workbook called `data/test-data.xlsx` is now the main repository for test data and flow configuration. It contains several sheets:

- `FlowConfig` — the master sheet that lists which spec runs in which order, which data sheet to use, and whether the step is enabled.
- Product sheets such as `TransferMaintenanceData`, `SavingsAccountData`, `CurrentAccountData`, `HPORDM`, `term_deposits`, and `topup_deposits`.
- Customer sheets such as `CRM` and `CRM_Mod` for retail and corporate CIF creation and modification.

3.2 Flow Configuration Sheet

The `FlowConfig` sheet controls the execution of end-to-end business flows. Each row contains:

- Flow name
- Step number
- Spec file to execute
- Data sheet to read
- Data row / instance number
- Customer type
- Enabled flag

This makes it possible to build a new flow by combining existing spec files and choosing which Excel data row each step should consume.

3.3 Resolver Scripts

Two resolver scripts were added to convert the Excel workbook into the JSON format the Playwright specs expect:

- `scripts/flowDataResolver.js` handles product data such as accounts, transfers, payment orders, and term deposits.
- `scripts/crmDataResolver.js` handles customer data, including flattening and inflating the nested CRM data structure.

These resolvers run just before the relevant spec, so the spec always receives clean, up-to-date data.

3.4 Ordered Runner and Menu

- `scripts/runOrdered.js` reads `FlowConfig`, executes each step in order, and passes data between specs through a shared-state file.
- `scripts/runFlowMenu.js` provides an interactive menu that lists the available enabled flows. The user picks a number and the selected flow runs.

3.5 Shared State Between Specs

A shared-state file allows values created in one step, such as a generated account ID or transaction ID, to be used by a later step. This is essential for end-to-end flows such as “create savings account → fund savings account → verify.”

4. Benefits of the Current Approach

Benefit	What it means for the business
Business ownership of test data	QA and business testers can change values in Excel without raising a development ticket.
Configurable test flows	Steps can be enabled, disabled, or reordered from the `FlowConfig` sheet. New flows can be assembled from existing specs.
Faster scenario creation	Creating a new test case is often as simple as adding a new row in Excel and pointing a flow step at it.
Reusable automation assets	One spec file supports many scenarios because it reads data at runtime.
Menu-driven execution	Non-technical users can select and run a flow from a simple numbered menu.
Versionable test data	The Excel workbook can be stored, diffed, and reviewed like any other business document.

5. Limitations of the Current Approach

Limitation	Business impact
Not fully no-code	Adding an arbitrary new Excel column does not automatically do anything. If the spec has not been told to read that column, a developer must still map it to a UI action.
One row = one scenario	Product sheets are flat. A single row represents one scenario. Multi-part cases, such as several debit and credit legs in one transaction, are not supported today.

CRM data is single-row per instance in the main workbook	Multiple addresses, contacts, currencies, or other-bank details cannot be added by simply inserting more rows in the main `test-data.xlsx` customer sheets today.
Excel formatting must be handled	Dates, blank cells, merged cells, and number formats can be misread by the automation unless the resolver is aware of them.
New screens and business logic need code	The framework is data-driven at the value level, not at the flow-construction level. New menus, new screens, and new validation rules still require page-object and spec development.

6. Client Requirements — Analysis and Recommendations

The client has raised five specific needs. The table below gives a quick summary; detailed guidance for each follows.

Requirement	Feasible today?	How to handle	Effort / impact
1. Add all application fields to Excel so the user can supply any field	**Partial**	Known fields can be added. New fields need a developer to wire them to the page object.	Low per field, but ongoing unless a generic mapper is built.
2. Many-to-many transactions (n debit accounts × m credit accounts)	**No as-is**	Restructure transfer data to support arrays of accounts/amounts and loop through them in the HTM page object.	Medium — changes data shape and transaction logic.
3. No generic `accountId` for deposit/withdrawal; use only the needed side	**Partial**	Update cash deposit/withdrawal specs to use `creditAccount` or `debitAccount` based on transaction type.	Small — mainly a data-naming and conditional logic change.
4. Multiple addresses, contacts, currencies, other-bank details via extra rows per instance	**Partial**	Extend the resolver to group rows that share the same `instance` number, similar to the existing retail Excel reader.	Medium — requires resolver and page-object updates.
5. Keep modification data on separate sheets	**Yes**	Continue using the dedicated `CRM_Mod` sheet. Do not merge with creation data.	None — recommended current practice.

6.1 Requirement 1 — Excel as a Mirror of Application Fields

What the client wants: All fields that appear in the Finacle application should be available as columns in the Excel workbook. The user should be able to fill any field and the automation should use it.

Current situation: Columns can be added freely to the Excel sheets, but the Playwright spec or page object must already know to read that column. Adding a column that is not referenced by the code has no effect.

Can it be done? Partially. For the set of fields already used by the existing page objects, it is straightforward to move them into Excel. For brand new fields, the page object must be updated to enter the value on the correct screen.

How to handle:

- Prepare a consolidated field catalogue with the business and QA teams listing every field the automation should support.
- Map each field to the screen, section, and action where it is entered.
- Develop the page-object method once for each new field, then expose it through the Excel column.

Recommendation: Do not promise a fully generic “any field without code” model unless the team is willing to build a field-to-UI mapping engine. The practical, low-risk path is a governed field list that grows over time as new page objects are added.

6.2 Requirement 2 — Many-to-Many Transactions

What the client wants: Under Transaction Maintenance, the client wants to post a single transaction with multiple debit accounts and multiple credit accounts. For example, three debit accounts and five credit accounts in one HTM transaction.

Current situation: The existing transfer specs post exactly one debit account and one credit account. The data row in the `TransferMaintenanceData` sheet contains one `debitAccount`, one `creditAccount`, and one `amount`.

Can it be done? Not with the current implementation, but it is technically feasible.

How to handle:

- Redesign the transfer data so it can hold a list of debit entries and a list of credit entries. This can be done by storing JSON arrays in one or more Excel cells, or by linking several rows to the same transaction instance.
- Update the HTM page object to add each debit part transaction and each credit part transaction in a loop.
- Ensure the amounts balance before posting, or capture and report the application error if they do not.

Recommendation: Treat this as a proper change request. It changes both the data model and the transaction logic. A design session should agree on how many legs are realistic and how imbalance is handled.

6.3 Requirement 3 — Remove Generic `accountId` for Deposits and Withdrawals

What the client wants: For a cash deposit, only the credit account should be supplied. For a cash withdrawal, only the debit account should be supplied. The generic `accountId` column should not be required.

Current situation: The cash deposit spec currently reads an `accountId` and a `partTranType` to know whether it is a credit. It does not distinguish between a debit and a credit account by column name.

Can it be done? Yes, with a small change.

How to handle:

- Introduce two explicit columns: `creditAccount` for deposits and `debitAccount` for withdrawals.
- Update the cash deposit/withdrawal specs to select the correct account based on the transaction type, menu code, or `partTranType`.
- For backward compatibility, keep `accountId` as a fallback until the new columns are fully adopted.

Recommendation: This is a quick, low-risk improvement that makes the data more intuitive. It should be done together with the menu code that identifies the transaction as deposit or withdrawal.

6.4 Requirement 4 — Multiple Addresses, Contacts, Currencies, and Other-Bank Details

What the client wants: The client wants to add more rows for the same customer instance and have the automation treat them as one record with multiple repeating sections, such as several addresses or several contacts.

Current situation: The main `test-data.xlsx` customer sheets expect one row per instance. The repository already contains an Excel reader in `tests/config/excelReader.ts` that supports multi-row grouping for another workbook, but that logic is not applied to the main customer resolver today.

Can it be done? Not out of the box, but the building blocks exist.

How to handle:

- Extend `scripts/crmDataResolver.js` to read all rows with the same `instance` number and group repeatable sections into arrays.

- Update the page objects that enter addresses, contacts, currencies, and other-bank details so they can loop through the arrays and add each entry.
- Decide whether the main workbook is the right place for this, or whether the retail data should continue to live in a separate workbook as it does today.

Recommendation: This is a medium-sized change. It should be planned carefully because it affects data shape, resolver logic, and screen interaction. A prototype for one repeatable section, such as contact details, is a good first step.

6.5 Requirement 5 — Separate Sheets for Creation and Modification Data

What the client wants: The client wants confirmation that maintaining separate sheets for creation and modification data is the right approach, rather than combining everything into one sheet.

Current situation: The workbook already uses a `CRM` sheet for creation and a `CRM_Mod` sheet for modification.

Can it be done? Yes, and it is already the current design.

How to handle:

- Continue with two dedicated sheets.
- Reserve the creation sheet for fields that are present at the time the record is created.
- Reserve the modification sheet for update-specific fields such as CIF ID, new values, and change reason.

Recommendation: Keep the separation. A single combined sheet would force every row to contain both creation and modification columns, creating sparse data, more errors, and harder maintenance. The current two-sheet model is the cleaner, safer choice.

7. Summary and Next Steps

The data-driven approach already delivers significant value: test data is centralised, flows are configurable, and non-technical users can run flows from a menu. The remaining client requirements are all addressable, but they vary in effort and risk.

Priority	Action	Rationale
1	Adopt `creditAccount` / `debitAccount` for cash deposits and withdrawals	Quick win; improves data clarity
2	Maintain separate creation and modification sheets	No change needed; continue current practice

3	Build a governed field catalogue for Requirement 1	Prevents scope creep and sets realistic expectations
4	Prototype multi-row support for one repeatable CRM section	Reduces risk before full Requirement 4 delivery
5	Design many-to-many transaction data shape and page-object flow	Largest change; needs a design session

By sequencing the work this way, the quick improvements can be delivered first while the more complex features are designed with the right level of input from business and QA.

This document is based on the current state of the repository and the data-driven approach as implemented in `data/test-data.xlsx`, `scripts/runOrdered.js`, `scripts/flowDataResolver.js`, `scripts/crmDataResolver.js`, and the associated Playwright specs.